

SOWMYA.KCODE

DSA DEMYSTIFIED

The Beginner's Step-by-Step Blueprint

Phone-First Multi-Language Master Edition

DESIGNED FOR MOBILE SCREENS

- Zero dense textbooks. Built for quick phone reading.
- One concept !' One visual !' Three implementations.
- & Java (Primary) | &™p C++ | Ø=Ü Python.
- Every problem explained with real intuition.
- 14 Master Interview Patterns demystified.
- 30, 60, & 90-Day Step-by-Step Study Roadmaps.

& JAVA

Primary Teaching

&™p C++

Fast Execution

Ø=Ü PYTHON

Clean Syntax

PERFECT FOR:

- ' Complete beginners scared of pointers and recursion.
- ' College students preparing for campus placements.
- ' Developers preparing for FAANG & product interviews.
- ' Working professionals switching to software roles.

Author: Sowmya Katkojwal • Instagram: @sowmya.kcode
Official Commercial Release • Price: ₹39

—	Foundations & Big-O Intuition (No Math Jargon)
—	Arrays & Dynamic Arrays (Lockers & Two Pointers)
Part 2	Strings & Character Manipulation (26 Buckets)
Part 3	Searching Algorithms (Binary Search 3-Rule Template)
Part 4	Sorting Algorithms (Elementary & Divide-and-Conquer)
Part 5	Linked Lists (Train Model, Pointers & Reversals)
Part 6	Stacks (LIFO Cafeteria Plates & Monotonic Stack)
Part 7	Queues & Deques (FIFO Line & BFS Traversal Engine)
Part 8	Hashing & HashMaps (Phone Directory & Two Sum)
Part 9	Recursion & Backtracking (Call Stack & Decision Trees)
Part 10	Trees & BST (Hierarchies, Traversals & Max Depth)
Part 11	Heaps & Priority Queues (ER Triage & Top-K Frequent)
Part 12	Graphs & Networks (Airports, Shortest BFS, Island DFS)
Part 13	Dynamic Programming (Remembering Answers, 1D & 2D)
Part 14	The 14 Master Interview Patterns Demystified
Part 15	30/60/90-Day Study Roadmaps & Interview Checklist

Ø=Ý5 CONCEPT: THE 5 QUESTIONS EVERY CHAPTER ANSWERS

1. WHAT is this? 2. WHY do we need it? 3. HOW does it work?
4. WHEN should I use it? 5. HOW do I recognize it in problems?

Ø=Ý5 CONCEPT: WHAT IS A DATA STRUCTURE?

A structured way to store items in computer memory so we can read and modify them efficiently, like books on organized shelves instead of a pile.

Big-O Without Scary Math

We measure operations growth as input size (n) increases:

Operations														
n²	%									/	O(n²)	[Slow: Nested Loops]		
n	%									/	O(n)	[Fair: Single Loop]		
log n	%	%	%	%	%	%	%	%	%		O(log n)	[Fast: Halving]		
1	%	%P	%P	%P	%P	%P	%P	%P	%P		O(1)	[Instant: Direct Jump]		
	%	%	%	%	%	%	%	%	%	%	%	%	%	% Input Size (n)

The 4 Core Complexities:

- 1. O(1) Constant: One direct action. Example: open locker #3.
- 2. O(log n) Logarithmic: Halving remaining items. Example: dictionary.
- 3. O(n) Linear: Scanning every item once. Example: attendance roll call.
- 4. O(n²) Quadratic: Nested loops. Example: all pairs shaking hands.

&j 1-PAGE REVISION: FOUNDATIONS

- O(1) is instant | O(log n) is halving | O(n) is a single loop | O(n²) is nested loops.
- Constants are dropped: O(2n) simplifies to O(n).
- Auxiliary Space: Memory used outside the original input array.

Ø=ß Warmup: Find Maximum Element

Assume arr[0] is max. Scan remaining elements and update champion.

& JAVA

```

public static int findMax(int[] arr) {
    int m = arr[0];
    for (int i = 1; i < arr.length; i++) if (arr[i] > m) m = arr[i];
    return m;
}

```

&™ C++

```

int findMax(const vector<int>& arr) {
    int m = arr[0];
    for (int i = 1; i < arr.size(); i++) if (arr[i] > m) m = arr[i];
    return m;
}

```

Ø=Ü PYTHON

```

def find_max(arr):
    m = arr[0]
    for i in range(1, len(arr)):
        if arr[i] > m: m = arr[i]
    return m

```

#ñ Complexity: Time $O(n)$ | Auxiliary Space $O(1)$

& p **Common Mistake** Initializing max = 0 (fails if all numbers are negative).

Ø=ß **Key Takeaway** To find an extreme in unsorted data, every item must be inspected once.

Ø=ß Move Zeroes to End (Scanning + Write Position)

& JAVA

```

public static void moveZeroes(int[] a) {
    int write = 0;
    for (int i = 0; i < a.length; i++) if (a[i] != 0) a[write++] = a[i];
    while (write < a.length) a[write++] = 0;
}

```

Q=4 Two Sum II — Sorted Array (LeetCode 167)
Sorted input allows O(n) converging two pointers instead of O(n²) nested loops.

```
& JAVA
public static int[] twoSum(int[] a, int target) {
    int l = 0, r = a.length - 1;
    while (l < r) {
        int s = a[l] + a[r];
        if (s == target) return new int[]{l + 1, r + 1};
        else if (s > target) r--; else l++;
    } return new int[]{};
}
```

```
& C++
vector<int> twoSum(const vector<int>& a, int t) {
    int l = 0, r = a.size() - 1;
    while (l < r) {
        int s = a[l] + a[r];
        if (s == t) return {l + 1, r + 1};
        else if (s > t) r--; else l++;
    } return {};
}
```

```
Python
def two_sum(a, t):
    l, r = 0, len(a) - 1
    while l < r:
        s = a[l] + a[r]
        if s == t: return [l + 1, r + 1]
        elif s > t: r -= 1
        else: l += 1
    return []
```

#ñ Complexity: Time O(n) | Auxiliary Space O(1)
& p Common Mistake Returning 0-based indices when the question asks for 1-based.
Q=4 Key Takeaway Sorted array + pair search -> consider Two Pointers immediately.

&j 1-PAGE REVISION: ARRAYS & DYNAMIC ARRAYS

- Read/Update: O(1) | Linear Search: O(n) | Dynamic Append: O(1) amortized.
- Last element index is always length - 1. Loop with i < arr.length.
- Two Pointers Converging: left = 0, right = n - 1 (for sorted pairs).
- Scanning + Write Index: in-place overwrite (e.g. Move Zeroes).

Ø=Ý5 CONCEPT: STRING IMMUTABILITY

In Java and Python, strings cannot be modified in-place. Repeated `str += "a"` creates garbage copies ($O(n^2)$). Use `StringBuilder` in Java!

The 26-Bucket Frequency Array

Because `'a' - 'a' = 0` and `'z' - 'a' = 25`, lowercase letters map directly to indices 0..25.

Char:	'a'	'b'	'c'	...	'z'
Index:	0	1	2	...	25
Formula:	char - 'a' (0(1) memory slot!)				

Ø=ßá Valid Anagram (LeetCode 242)

```
& JAVA
public static boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < s.length(); i++) {
        count[s.charAt(i) - 'a']++;
        count[t.charAt(i) - 'a']--;
    }
    for (int c : count) if (c != 0) return false;
    return true;
}
```

```
Ø=Ü PYTHON
def is_anagram(s, t):
    if len(s) != len(t): return False
    count = [0] * 26
    for i in range(len(s)):
        count[ord(s[i]) - 97] += 1
        count[ord(t[i]) - 97] -= 1
    return all(c == 0 for c in count)
```

#ñ Complexity: Time $O(n)$ | Auxiliary Space $O(1)$ (26 slots)

- & p Common Mistake: Using `==` instead of `.equals()` in Java for string comparison.
- Ø=ßá Key Takeaway: Frequency counting with fixed `int[26]` gives $O(1)$ space.

Ø=Ý5 CONCEPT: LINEAR VS BINARY SEARCH

Linear search ($O(n)$) checks every item. Binary search ($O(\log n)$) cuts half of remaining candidates every guess. Requires sorted input!

Target = 7 in sorted array:

[1, 3, 5, 7, 9, 11, 13] low=0, high=6, mid=3 (val 7)
 %2

FOUND in 1 step! (1000 items take only 10 steps!)

The 3 Invariants of Binary Search:

1. low = 0, high = n - 1
2. mid = low + (high - low) / 2 (Prevents integer overflow!)
3. while (low <= high)

& JAVA

```
public static int binarySearch(int[] a, int target) {
    int low = 0, high = a.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid;
        else if (a[mid] < target) low = mid + 1;
        else high = mid - 1;
    } return -1;
}
```

Ø=Ü PYTHON

```
def binary_search(a, target):
    low, high = 0, len(a) - 1
    while low <= high:
        mid = low + (high - low) // 2
        if a[mid] == target: return mid
        elif a[mid] < target: low = mid + 1
        else: high = mid - 1
    return -1
```

#ñ Complexity: Time $O(\log n)$ | Auxiliary Space $O(1)$

& p Common Mistake: Writing mid = (low + high) / 2 which overflows for large numbers.

Ø=ßä Key Takeaway: Binary search cuts search space by 50% each step.

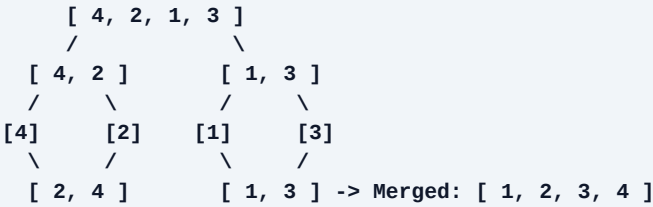
Ø=Ý5 CONCEPT: WHY SORT?

Searching an unsorted list takes $O(n)$. Once sorted, searching takes blazingly fast $O(\log n)$ time.

Bubble Sort (Bubbling largest item right):

```
[ 5, 3, 4, 1 ]
5 > 3 -> swap -> [ 3, 5, 4, 1 ]
5 > 4 -> swap -> [ 3, 4, 5, 1 ]
5 > 1 -> swap -> [ 3, 4, 1, 5 ] (5 reached end!)
```

Divide & Conquer: Merge Sort Tree



&i 1-PAGE REVISION: SORTING

- Bubble/Insertion/Selection: $O(n^2)$ time, $O(1)$ space.
- Merge Sort: $O(n \log n)$ guaranteed, requires $O(n)$ auxiliary memory.
- Quick Sort: $O(n \log n)$ average, in-place $O(\log n)$ call stack.
- In interviews: Use built-in `Arrays.sort()` unless asked for implementation.

Stack: Cafeteria Plate Dispenser (LIFO)

TOP % % %° [30] (Last added, First out!)
[20]
[10] push/pop/peek: 0(1)

Queue: Grocery Checkout Line (FIFO)

FRONT % % %° [10] [20] [30] %Ä% % REAR
(Served first) (Joined last)

Ø=Ý4 Valid Parentheses (LeetCode 20)

& JAVA
public static boolean isValid(String s) {
 ArrayDeque<Character> st = new ArrayDeque<>();
 for (char c : s.toCharArray()) {
 if (c == '(') st.push('(');
 else if (c == '{') st.push('{');
 else if (c == '[') st.push('[');
 else if (st.isEmpty() || st.pop() != c) return false;
 } return st.isEmpty();
}

Ø=Ü PYTHON

def is_valid(s):
 st = []
 m = {"(": ")", "{": "}", "[": "]" }
 for c in s:
 if c in m: st.append(m[c])
 elif not st or st.pop() != c: return False
 return len(st) == 0

&j 1-PAGE REVISION: STACKS & QUEUES

- Stack (LIFO): push/pop O(1) via ArrayDeque in Java, append/pop in Python.
- Queue (FIFO): offer/poll O(1) via LinkedList in Java, deque in Python.
- Monotonic Stack: Finds next greater / smaller elements in O(n).

Ø=Ý5 CONCEPT: THE PHONE CONTACT BOOK

Searching an array takes $O(n)$. A HashMap calculates a hash code for any key to access its value in $O(1)$ average time!

Key: "Alice" % % %° Hash Code % % %° Bucket 3 % % %° Value: 98765
Key: "Bob" % % %° Hash Code % % %° Bucket 1 % % %° Value: 12345

Ø=Ý4 Two Sum — Unsorted (LeetCode 1)

Scan array: does map already hold (target - num)? If yes, done!

```
& JAVA
public static int[] twoSum(int[] a, int t) {
    HashMap<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        int comp = t - a[i];
        if (map.containsKey(comp)) return new int[]{map.get(comp), i};
        map.put(a[i], i);
    } return new int[]{};
}
```

```
&™p C++
vector<int> twoSum(const vector<int>& a, int t) {
    unordered_map<int, int> m;
    for (int i = 0; i < a.size(); i++) {
        int comp = t - a[i];
        if (m.count(comp)) return {m[comp], i};
        m[a[i]] = i;
    } return {};
}
```

```
Ø=Ü PYTHON
def two_sum(a, t):
    seen = {}
    for i, n in enumerate(a):
        comp = t - n
        if comp in seen: return [seen[comp], i]
        seen[n] = i
    return []
```

#ñ Complexity: Time $O(n)$ | Auxiliary Space $O(n)$

& p Common Mistake Assuming HashMap is sorted. Use TreeMap in Java if keys must stay in order.
Ø=ßã Key Takeaway Lookup & insert are $O(1)$ average. Perfect for complement matching.

Ø=Ý5 CONCEPT: RUSSIAN DOLLS

A function calling itself with a smaller version of the problem until it hits the Base Case (the solid baby doll).

Calling factorial(3):

```

fact(3) calls 3 * fact(2)
  fact(2) calls 2 * fact(1)
    fact(1) returns 1 (Base Case!)
  fact(2) returns 2 * 1 = 2
fact(3) returns 3 * 2 = 6

```

Backtracking: Choose -> Explore -> Unchoose

& JAVA SUBSETS

```

public static void backtrack(int[] nums, int start, List<Integer> cur,
List<Integer> res){
    if(start == nums.length){
        res.add(new ArrayList<>(cur));
    }
    for (int i = start; i < nums.length; i++) {
        cur.add(nums[i]);           // 1. Choose
        backtrack(nums, i + 1, cur, res); // 2. Explore
        cur.remove(cur.size() - 1); // 3. Unchoose (Backtrack!)
    }
}

```

&i 1-PAGE REVISION: RECURSION

- Must have: 1. Base Case (stops stack overflow), 2. Recursive step.
- Recursion Call Stack consumes auxiliary memory equal to max recursion depth.
- Backtracking = DFS on a decision tree with undoing choices.

Binary Search Tree (Left < Root < Right)

10

/ \

5 15

(SORTED!)

2 7

Traversals:

• Preorder (Root, L, R) : 10, 5, 2, 7, 15

• Inorder (L, Root, R) : 2, 5, 7, 10, 15

• Postorder (L, R, Root) : 2, 7, 5, 15, 10

Ø=ßá Maximum Depth of Tree (LeetCode 104)

& JAVA

```
public static int maxDepth(TreeNode r) {
    if (r == null) return 0;
    return 1 + Math.max(maxDepth(r.left), maxDepth(r.right));
}
```

Heap / Priority Queue (ER Hospital Triage)

Min-Heap: Smallest item at root (Peek O(1), Insert/Remove O(log n))

Max-Heap: Largest item at root (PriorityQueue<Integer>((a,b) -> b - a))

Ø=Ý4 Top K Frequent Elements (LeetCode 347)

Keep a Min-Heap of size k. Discard smallest frequency when size exceeds k!

Ø=Ý5 CONCEPT: AIRPORTS & FLIGHTS

Vertices (nodes) connected by Edges (lines). Adjacency List stores each node's neighbor list.

A % % % % % % %	B	BFS: Explores layer-by-layer using Queue.
%	%	Guarantees shortest path in unweighted graphs!
%	%	
C % % % % % % %	D	DFS: Dives deep using Stack / Recursion.

Ø=Ý4 Number of Islands (LeetCode 200)

```
& JAVA
public static int numIslands(char[][] g) {
    int count = 0;
    for (int r = 0; r < g.length; r++)
        for (int c = 0; c < g[0].length; c++)
            if (g[r][c] == '1') { count++; dfs(g, r, c); }
    return count;
}
static void dfs(char[][] g, int r, int c) {
    if (r<0||c<0||r>=g.length||c>=g[0].length||g[r][c]!='1') return;
    g[r][c] = '0'; // Sink cell
    dfs(g,r+1,c); dfs(g,r-1,c); dfs(g,r,c+1); dfs(g,r,c-1);
}
```

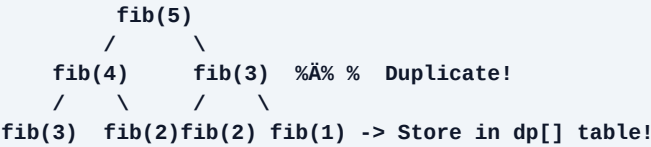
&j 1-PAGE REVISION: GRAPHS

- Adjacency List: List<List<Integer>> with O(V + E) memory.
- BFS = Shortest path (Queue) | DFS = Connected components (Recursion).
- Always track visited nodes to avoid infinite cycles!

Ø=Ý5 CONCEPT: THE GOLDEN DP RULE

Dynamic Programming is simply remembering answers to subproblems so you never re-calculate them. Think of it as recursion + a memory notebook.

Duplicate work in naive Fibonacci:



Ø=ßÁ Climbing Stairs (LeetCode 70)

To reach step n, jump from n-1 or n-2 -> $dp[n] = dp[n-1] + dp[n-2]$

& JAVA (O(1) SPACE)

```
public static int climbStairs(int n) {
    if (n <= 2) return n;
    int p2 = 1, p1 = 2;
    for (int i = 3; i <= n; i++) {
        int cur = p1 + p2; p2 = p1; p1 = cur;
    } return p1;
}
```

Ø=Ü PYTHON

```
def climb_stairs(n):
    if n <= 2: return n
    p2, p1 = 1, 2
    for _ in range(3, n + 1):
        cur = p1 + p2
        p2, p1 = p1, cur
    return p1
```

&i 1-PAGE REVISION: DYNAMIC PROGRAMMING

- 1. Define State: What does $dp[i]$ mean in English?
- 2. Base Case: Smallest known answers ($dp[0]$, $dp[1]$).
- 3. Recurrence: Build $dp[i]$ using previous subproblems.
- 4. Space Optimization: Replace array with 2 variables if only past 2 matter.

: Sorted arrays, pairs, in-place palindrome check.
: Linked list cycle detection, finding list middle.
: Contiguous subarray/substring max/min limits.

- 4. **Merge Intervals**: Overlapping schedules, meeting rooms.
- 5. **Cyclic Sort**: Arrays with numbers in range 1..n or 0..n.
- 6. **In-Place Reversal**: Reversing linked list sublists without extra allocation.
- 7. **Tree BFS**: Level-order traversal, zigzag order using Queue.
- 8. **Tree DFS**: Root-to-leaf path sums, diameter, max depth.
- 9. **Two Heaps**: Finding running median of a data stream.
- 10. **Subsets / Backtrack**: Power sets, permutations, combinations.
- 11. **Modified Binary Search**: Rotated sorted arrays, boundary searching.
- 12. **Top K Elements**: Min-Heap of size k for largest elements.
- 13. **K-Way Merge**: Merging k sorted lists using PriorityQueue.
- 14. **0/1 Knapsack & DP**: Pick or skip items to maximize value.

🚀 The 30-Day Beginner Action Plan

- Week 1: Big-O, Arrays, Two Pointers, Strings (Parts 0..2)
- Week 2: Binary Search, Sorting, Linked Lists (Parts 3..5)
- Week 3: Stacks, Queues, HashMaps (Parts 6..8)
- Week 4: Recursion, Trees, Graphs, Basic DP (Parts 9..13)

📅 The 6-Step Live Interview Playbook

1. Clarify Constraints (1 min): Empty input? Negative numbers? Duplicates?
2. Propose Brute Force (2 min): State $O(n^2)$ baseline before jumping to code.
3. Name the Pattern (2 min): "Because array is sorted, Two Pointers drops this to $O(n)$."
4. Dry Run on Small Example (2 min): Trace indices on scratchpad first!
5. Write Clean Code (10 min): Clear variable names, handle boundaries.
6. Test Edge Cases (3 min): 0 elements, 1 element, all duplicates.



